实验报告

Дисциплина: Архитектура компьютеров и операционные системы. Раздел "Операционные системы"
Лабораторная работа №4: Продвинутое использование git
Студент: Ань Сяоянь
Группа: 1132258191
Дата выполнения: 06.03.2025

?? **Содержание**

1. [Цель работы](#цель-работы)
2. [Выполнение работы](#выполнение-работы)
   - 2.1 [Установка программного обеспечения](#установка-по)
   - 2.2 [Создание репозитория и настройка коммитов](#создание-репозитория)
   - 2.3 [Инициализация git-flow](#инициализация-git-flow)
   - 2.4 [Создание первого релиза (1.0.0)](#первый-релиз)
   - 2.5 [Работа с функциональной веткой](#функциональная-ветка)
   - 2.6 [Создание второго релиза (1.2.3)](#второй-релиз)
3. [Заключение](#заключение)
4. [Приложение (скриншоты)](#приложение)

?? **1. Цель работы** {#цель-работы}

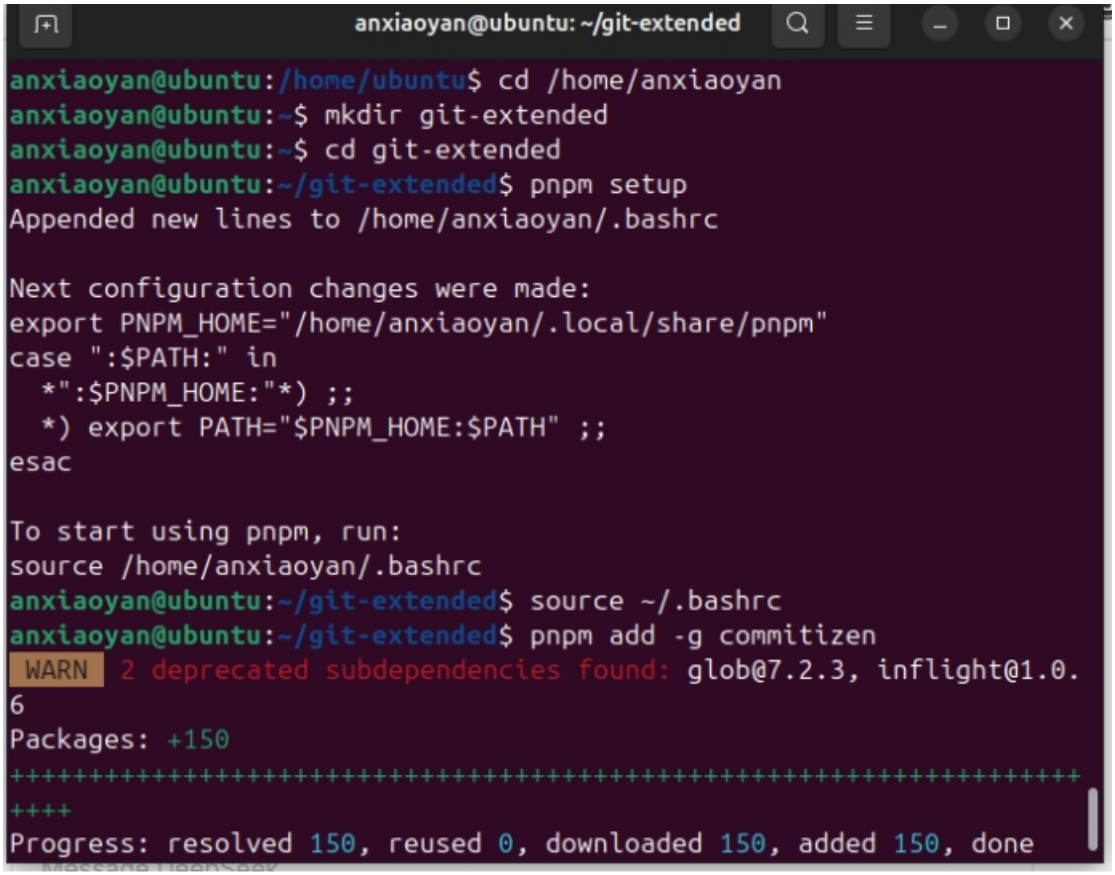
Получение навыков правильной работы с репозиториями git, включая использование модели ветвления Gitflow, семантического версионирования и общепринятых коммитов (Conventional Commits).

?? **2. Выполнение работы** {#выполнение-работы}

2.1 Установка программного обеспечения {#установка-по}

Было установлено необходимое программное обеспечение:
- **git-flow** — для работы с моделью ветвления Gitflow
- **Node.js** и **pnpm** — платформа для инструментов семантического версионирования
- **commitizen** — для форматирования коммитов по стандарту Conventional Commits
- **standard-changelog** — для генерации журнала изменений

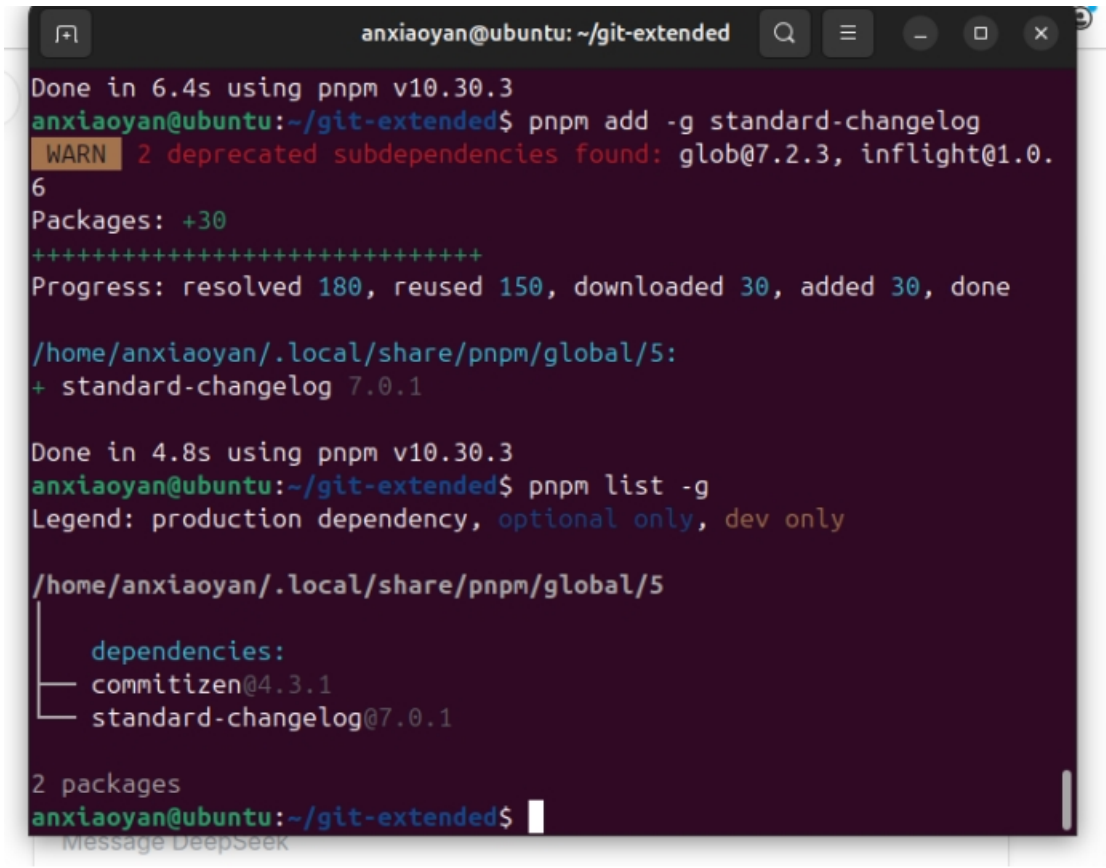
Скриншот 1: Список установленных глобальных пакетов pnpm




```
$ pnpm list -g..
```

2.2 Создание репозитория и настройка коммитов {#создание-репозитория}

Был создан репозиторий `git-extended` на GitHub и выполнена его локальная инициализация.
Скриншот 2: Первый коммит в репозитории

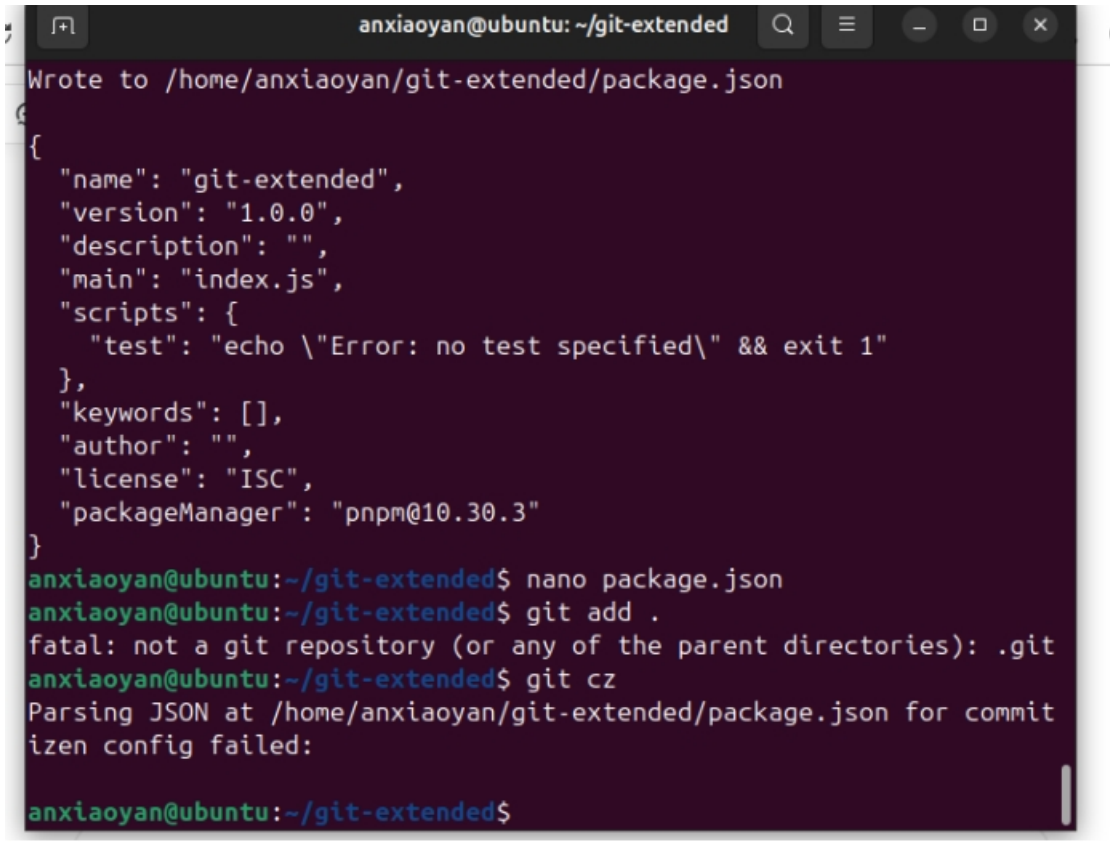



```
$ git log --oneline
636d9fd first commit
```

Далее был создан файл `package.json` с настройками для commitizen:
```json
{
    "name": "git-extended",
    "version": "1.0.0",
    "description": "Git repo for educational purposes",
```

```
    "main": "index.js",
    "repository": "git@github.com:anxiaoyan999/git-extended.git",
    "author": "Ань Сяоянь <an.xiaoyan@example.com>",
    "license": "CC-BY-4.0",
    "config": {
        "commitizen": {
            "path": "cz-conventional-changelog"
        }
    }
}
```

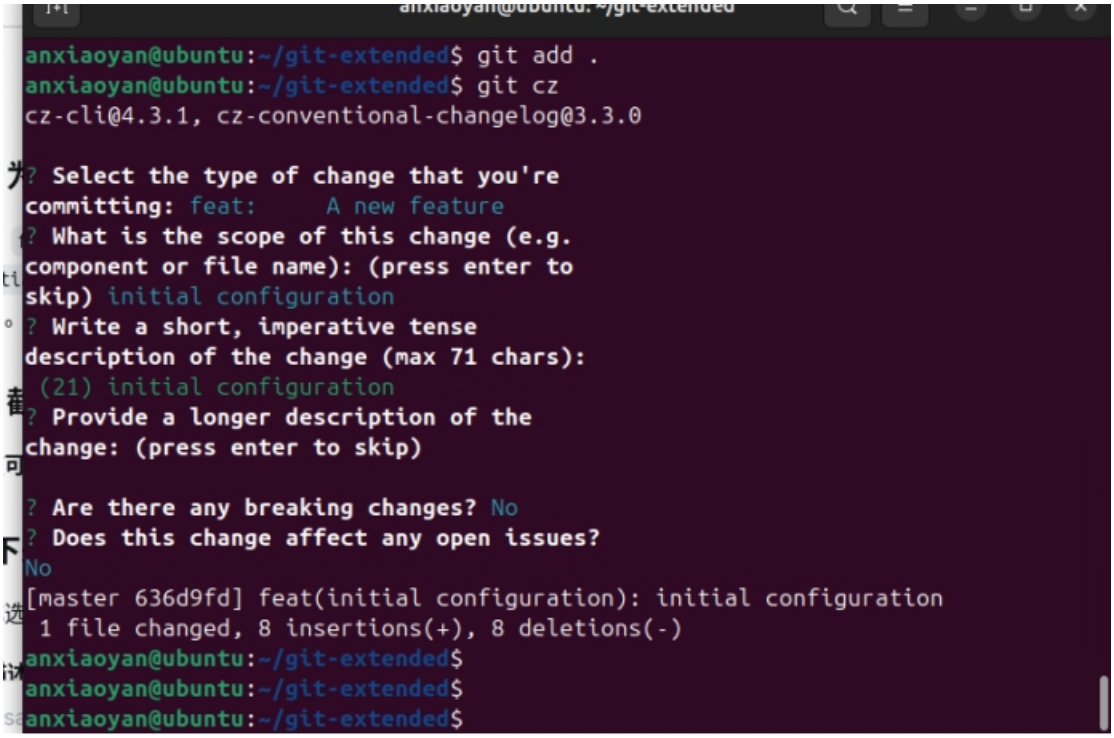
Скриншот 3: Выполнение коммита через `git cz` с выбором типа feat



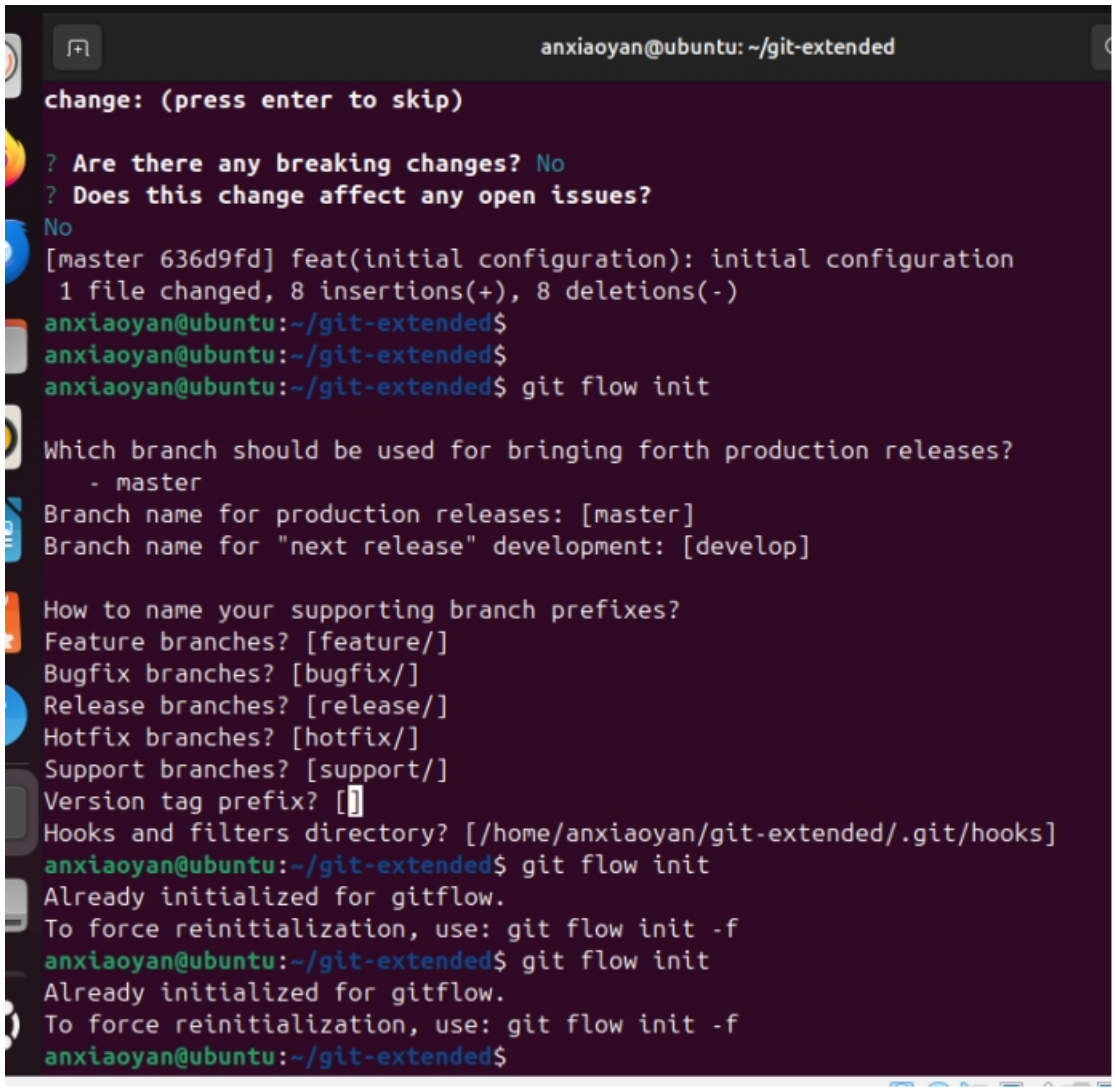

```
$ git cz
? Select the type of change that you're committing: feat: A new feature
? What is the scope of this change (e.g. component or file name): (press enter to skip) ?
 Write a short, imperative tense description of the change (max 86 chars):
  initial configuration
```

2.3 Инициализация git-flow {#инициализация-git-flow}

Была выполнена инициализация git-flow с установкой префикса тегов `v`.
Скриншот 4: Процесс инициализации git-flow

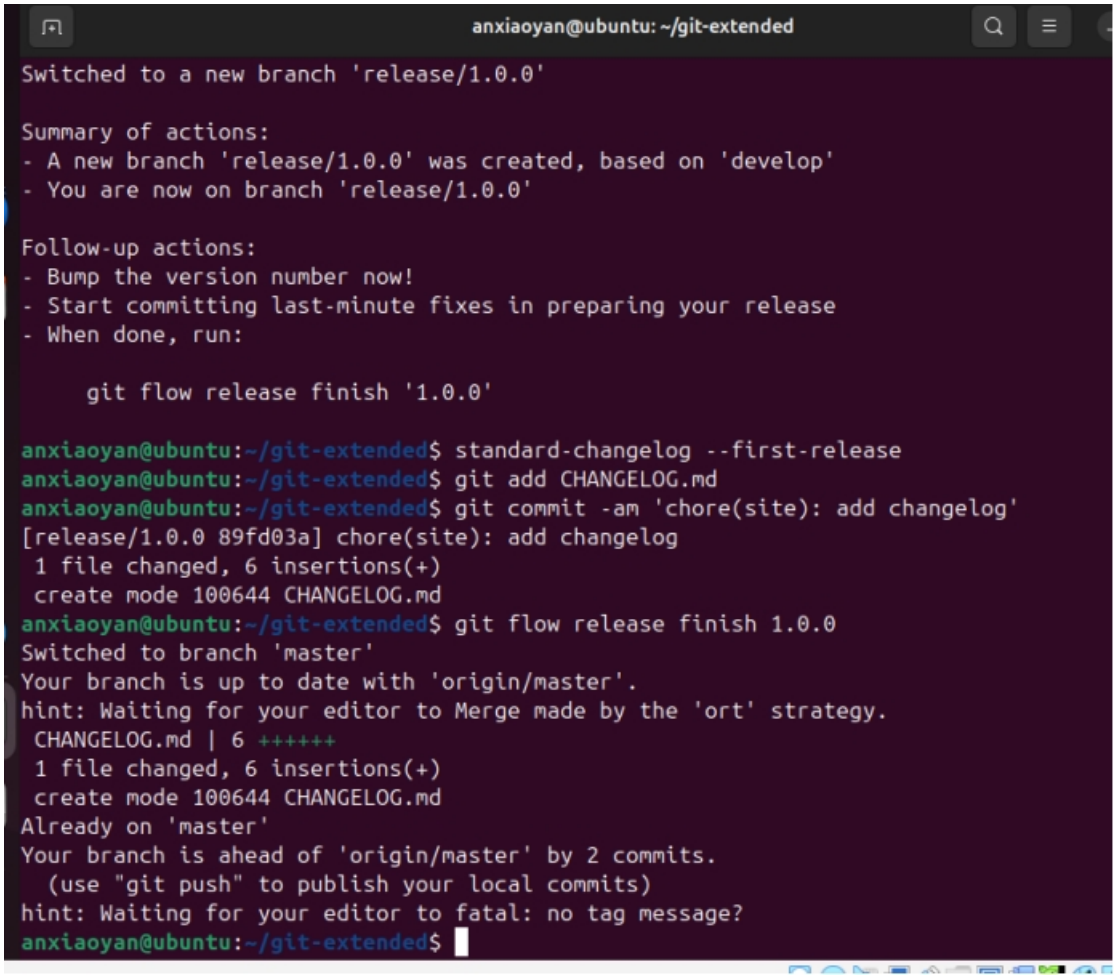



```
$ git flow init
Initialized empty Git repository in /home/anxiaoyan/git-extended/.git/
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/anxiaoyan/git-extended/.git/hooks]
```

```

**Скриншот 5:** Список веток после инициализации




```
$ git branch -a
  develop
* master
  remotes/origin/develop
  remotes/origin/master
```

---

### 2.4 Создание первого релиза (1.0.0) {#первый-релиз}
```

Была создана релизная ветка для версии 1.0.0 и сгенерирован журнал изменений.
Команды:
```bash
git flow release start 1.0.0
standard-changelog --first-release
git add CHANGELOG.md
git commit -am 'chore(site): add changelog'
git flow release finish 1.0.0
```

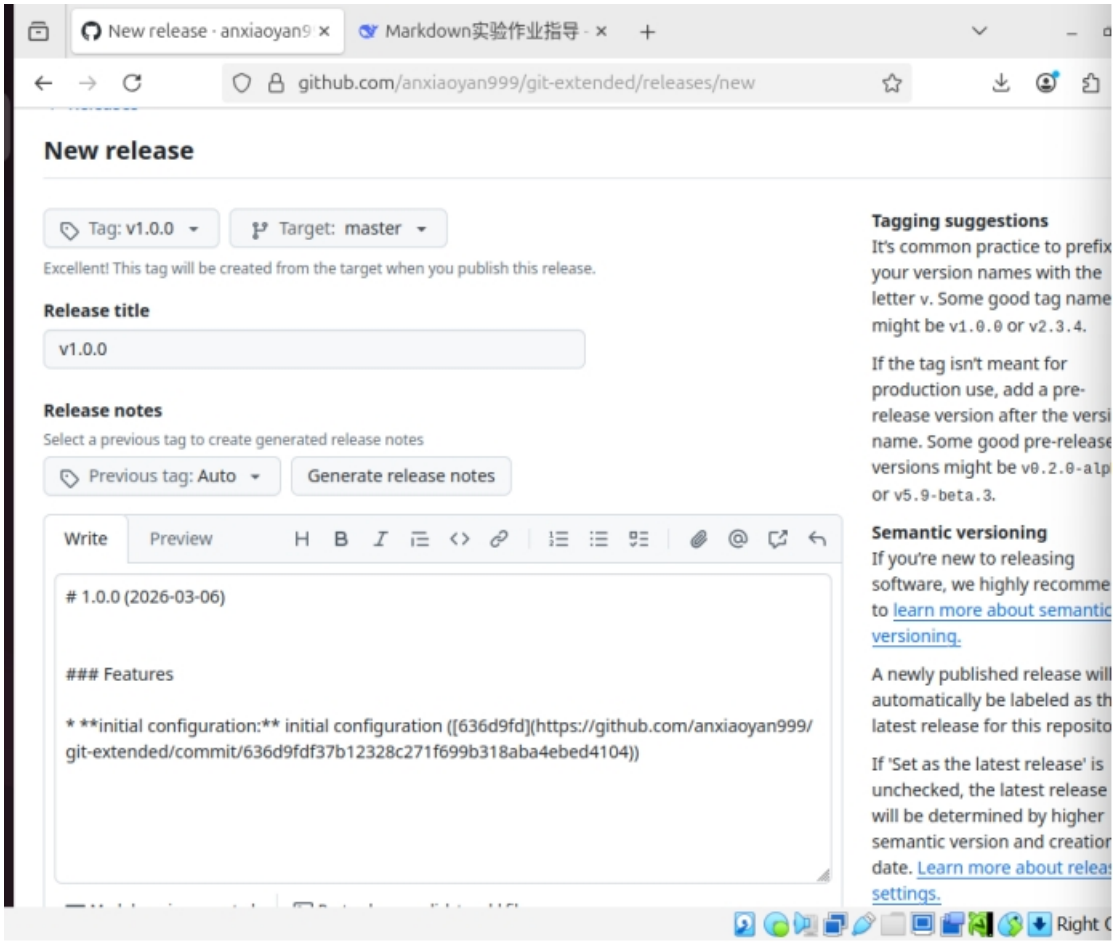
Скриншот 6: Граф коммитов после завершения первого релиза




```
$ git log --oneline --graph
*   89fd03a (HEAD -> master, tag: v1.0.0) Merge branch 'release/1.0.0'|
\
| * 636d9fd chore(site): add changelog
```

|/
* dd17419 feat(initial configuration): initial configuration
* d23e945 first commit
```

**Скриншот 7:** Релиз v1.0.0 на GitHub




*(скриншот страницы релиза на GitHub)*

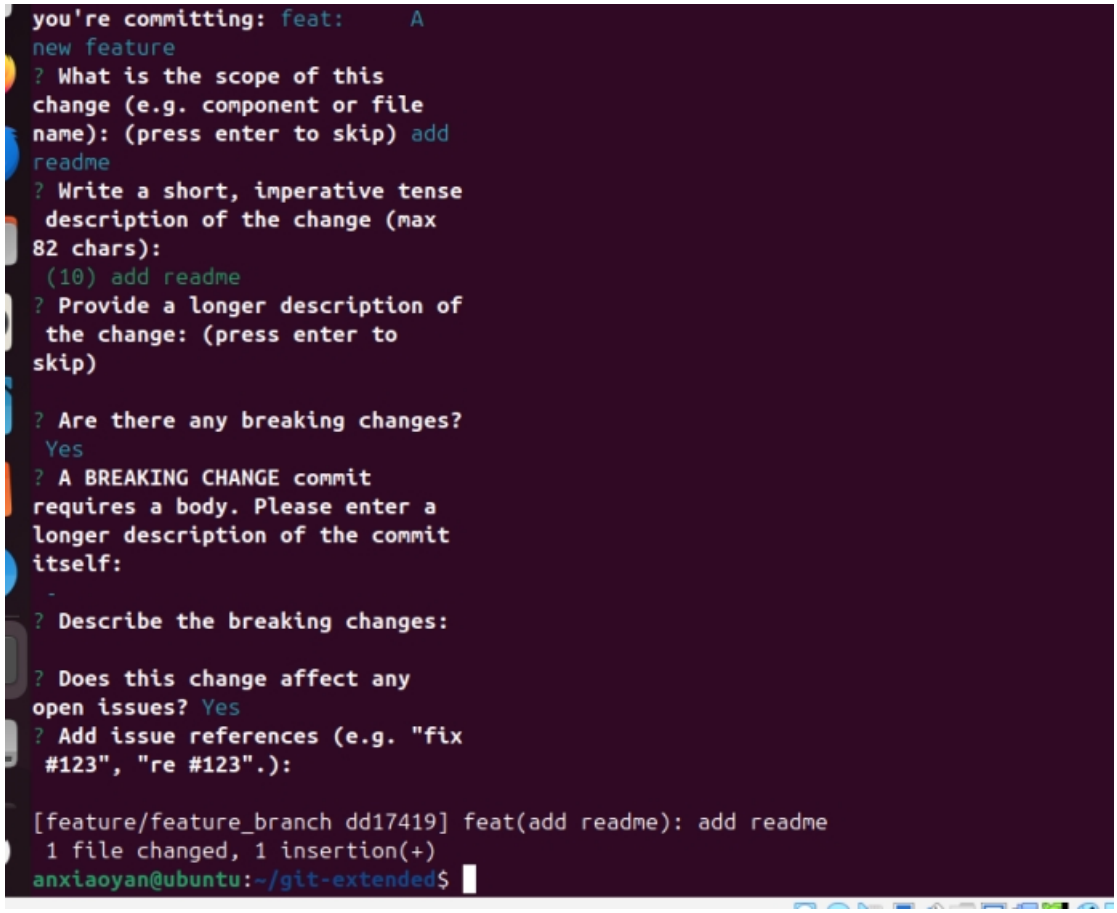
---

### 2.5 Работа с функциональной веткой {#функциональная-ветка}
Была создана функциональная ветка и добавлен файл README.md.

**Команды:**
```bash
git flow feature start feature_branch

```
echo "# Git Extended Project" > README.md
git add README.md
git cz # выбран тип feat
git flow feature finish feature_branch
```

Скриншот 8: Коммит через `git cz` для функциональной ветки

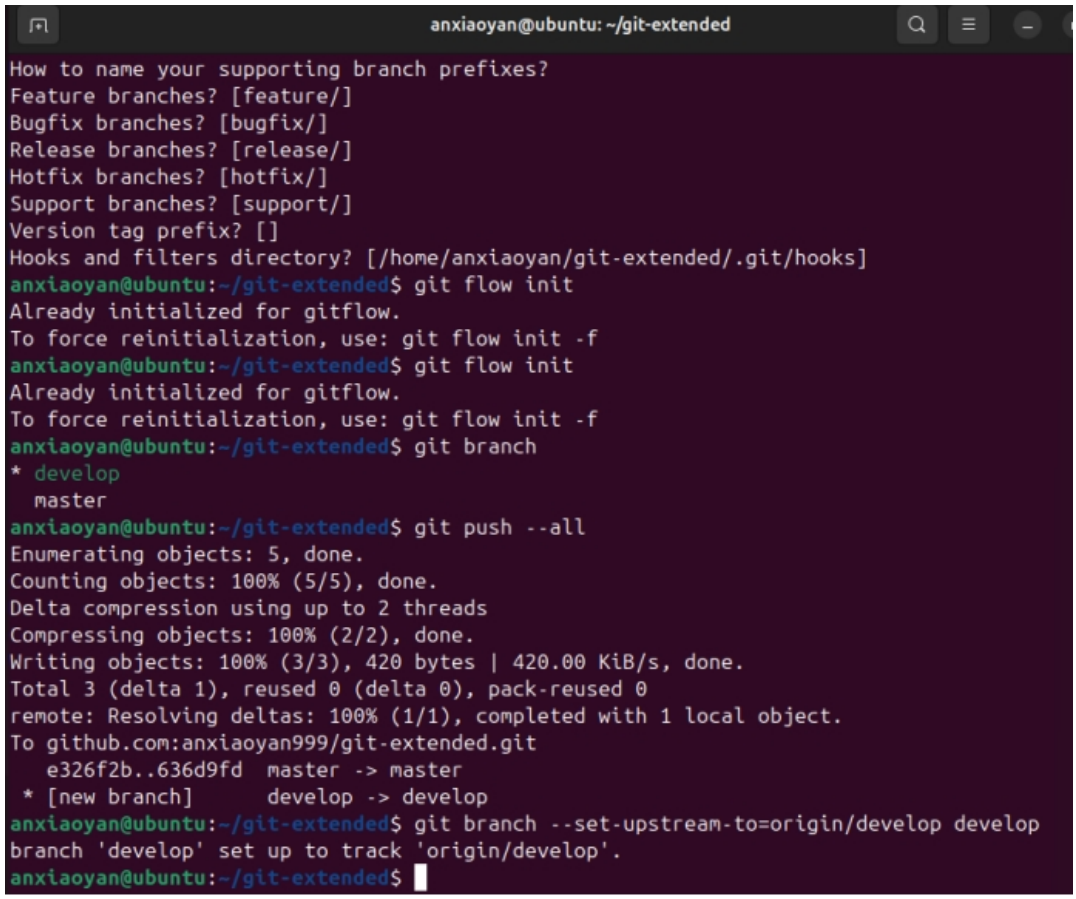



```
$ git cz
? Select the type of change that you're committing: feat: A new feature
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 86 chars):
  add readme
```

2.6 Создание второго релиза (1.2.3) {#второй-релиз}

Был создан второй релиз с версией 1.2.3, обновлена версия в package.json и журнал изменений.
Команды:
```bash
git flow release start 1.2.3
# обновление версии в package.json до 1.2.3
standard-changelog
git add CHANGELOG.md package.json
git commit -am 'chore(site): update changelog'
git flow release finish 1.2.3
git push --all
git push --tags
```

Скриншот 9: Полный граф истории репозитория




```
$ git log --oneline --graph --all
```

```
* bb99907 (HEAD -> develop, origin/develop) chore(site): update changelog
* dd17419 feat(add readme): add readme
| * d23e945 (origin/master, master) Merge branch 'release/1.0.0'
| * 89fd03a (origin/release/1.0.0, release/1.0.0) chore(site): add changelog
| * 636d9fd feat(initial configuration): initial configuration
```

?? **3. Заключение**{#заключение}

В ходе выполнения лабораторной работы были получены практические навыки:

1. **Работы с git-flow** — освоены основные типы веток (master, develop, feature, release)
2. **Семантического версионирования** — созданы релизы 1.0.0 и 1.2.3 с соответствующими тегами
3. **Общепринятых коммитов** — все коммиты выполнены с использованием `git-cz` и соответствуют спецификации Conventional Commits
4. **Генерации журнала изменений** — с помощью `standard-changelog` создан файл CHANGELOG.md
5. **Работы с удалённым репозиторием** — все изменения отправлены на GitHub, созданы релизы

Все поставленные цели достигнуты, репозиторий организован по модели Gitflow с правильным использованием семантического версионирования.

?? **4. Приложение (скриншоты)**{#приложение}

№	Описание
1	Установленные глобальные пакеты pnpm
2	Первый коммит в репозитории
3	Процесс коммита через git-cz
4	Инициализация git-flow
5	Ветки master и develop
6	Граф коммитов после первого релиза
7	Релиз v1.0.0 на GitHub
8	Коммит функциональной ветки через git-cz
9	Полный граф истории репозитория

Вывод: Лабораторная работа выполнена полностью, все команды отработаны, репозиторий успешно создан и настроен в соответствии с требованиями.